

L2 Knowledge Graph - Understanding

What the L2 layer is, and how it fits the plan

Layer L2 - Knowledge Graph | PostgreSQL 16 + Apache AGE 1.6.0

Prototype Jun 10 - Prod Jun 15 - Doc due Jun 12 (async review)

1. What L2 is, in one paragraph

L2 is the Knowledge Graph layer. L1 (SignalFabric) streams in raw records - crew, contracts, vessel/port data, certificates. On their own these are disconnected rows. L2's job is to weave them into ONE connected graph of nodes and relationships, so that the smarter layers above (L3 Intelligence, L4 Decision) can answer questions by walking relationships instead of writing big multi-table joins. The graph lives inside the same PostgreSQL database using the Apache AGE extension - no separate graph database is added.

2. Where L2 sits

```
L1 SignalFabric    -> streams raw events (crew, contracts, vessels, certs)
L2 Knowledge Graph-> connects them into a graph <-- THIS LAYER
L3 Intelligence    -> agents traverse the graph to match & rank crew
L4 Decision Graph  -> records every decision as a trace for learning
```

L2 is the memory of the system: the shared, connected picture every layer above reads from.

3. The three dimensions of the graph

The plan describes L2 as ONE graph seen through three lenses ("dimensions"). They share the same nodes; each lens adds its own relationships.

Dimension	Answers the question	Example path	Status
EntityMap	What exists and how is it factually related?	Crew -HOLDS-> Certificate	BUILT
OpsMap	What is the operational state right now?	sign-off -> search -> match -> onboard	Planned
OrgMap	How is the organisation structured?	Company -> Fleet -> Vessel -> Rank	Planned

This deliverable builds Dimension 1 (EntityMap) in full. OpsMap and OrgMap are specified as a baseline plan for the Jun 12 review; they reuse EntityMap's nodes.

4. EntityMap - the five things the domain is made of

- > **Crew** - a seafarer, either a sign-on candidate or someone onboard.
- > **Vessel** - a ship.
- > **Port** - a port where crew are, or a ship calls.
- > **Certificate** - a qualification a seafarer holds (e.g. STCW, GMDSS).
- > **Contract** - one engagement of a crew member on a vessel.

These connect through plain-English relationships: a Crew **HOLDS** a Certificate, is **ASSIGNED_TO** a Vessel, is **CURRENTLY_AT** a Port, and **SIGNED** a Contract; a Vessel **CALLS_AT** a Port; a Contract is **FOR_VESSEL** and **AT_PORT**.

5. What "done" means (the plan's exit criteria)

Exit criterion (from the plan)	How L2 meets it
All 3 dimensions populated	EntityMap loaded (90 nodes, 189 edges); Ops/Org specced
Crew search by rank + cert + port works	Graph search endpoint returns correct crew
Full relationship traversal works	Traversal endpoint walks multi-hop paths
Test UI query < 3 seconds	Measured 5-37 milliseconds
No data duplication across dimensions	Each real-world thing is exactly one node

6. Why a graph (and why Postgres + AGE)

- > **A graph fits the questions.** "Which available Chief Officer at Rotterdam holds GMDSS and can replace the one signing off?" is a path through relationships, not a row filter.
- > **One database, not two.** AGE runs the graph **INSIDE** the existing PostgreSQL, so graph context and relational records share one connection - simpler ops, no data sync.
- > **Standard query language.** AGE speaks openCypher, the widely-used graph query language, so the intelligence layer reads familiar queries.
- > **Swappable backend.** A 'fallback' mode builds the same shape in plain Python when AGE isn't present, so the feature always demos; 'age' mode runs the real graph.

7. What the team gets from L2

- > **For L3 (Intelligence):** a ready-made web of facts to traverse instead of re-joining tables.
- > **For reviewers:** a clear schema, a working EntityMap, and a concrete plan for the other two maps.
- > **For operations:** fast (<3s) answers to crew-matching questions backed by real relationships.